From Lean Proofs to Public Claims

Release checks and trust boundaries in one formal mathematics repository

Will Cook

July 2026

Abstract

Lean verifies that a proof establishes the formal statement written in the source. It does not check that a README or paper describes that statement faithfully. We study that second problem in one public Lean repository. This is an architecture and experience report about preserving reviewed public-claim boundaries, not an empirical evaluation of reviewer performance. For each selected result, a mathematician records the public wording, the formal statement said to support it, the range actually proved, and the adjacent stronger statement that remains open. The release workflow checks that later edits preserve those recorded relationships; it does not decide whether the original mathematical judgement was correct.

The worked example separates a finite theorem from an open problem. Lean has checked one finite certificate at every lcm-diagonal scale $t \leq 82$. The open requirement asks for such certificates beyond every fixed cutoff, and the repository proves that this unbounded supply is equivalent to an affirmative answer to Erdős Problem 249. Thus the finite theorem is not a partial wording of the open conclusion: the two have different logical forms. A historical README edit erased the distinction and passed because that relationship had not been registered. After registration, a deliberately false copy was rejected. This establishes one failure and repair, not complete claim discovery, correct interpretation, or a solution of either problem.

1 The publication gap

The repository studied here is a public development in the Lean 4 proof assistant [1] around two unsolved problems in number theory, Erdős Problems 249 and 257. Both problems remain open. The project proves intermediate results, exact reformulations, and finite certificates around them; it does not claim a solution to either problem.

Lean is both a language for writing mathematics precisely and a program for checking proofs. A Lean theorem has a formal statement and a proof object. Its *kernel*, the small trusted part of the system, checks the proof using the file's definitions, earlier theorems, and explicit assumptions [1, 4]. Acceptance is therefore a strong conclusion about the formal statement. A person must still judge whether that statement expresses the intended mathematics and whether later prose says anything stronger.

Authorship and AI use. The prose, formal proofs, and software in this version were drafted and revised by large-language-model agents under Will Cook's direction. Cook set the objectives and acceptance criteria, reviewed and authorised the manuscript, and is responsible for its contents. The agents are production tools, not authors. See Appendix A.

$$\underbrace{\forall t \le 82,\ \mathrm{Cert}(t)}_{\text{checked finite theorem}} \qquad \not\Longrightarrow \qquad \underbrace{\forall T,\ \exists t > T,\ \mathrm{Cert}(t)}_{\text{open requirement}}.$$

$\mathrm{Cert}(t)$ means that finite data lets Lean establish one stated non-integrality result at scale t. The right-hand statement asks for a witness after every cutoff. No fixed upper bound has that logical form.

Figure 1: The distinction the release design is built to preserve. The development proves that the open statement on the right is equivalent to the irrationality claim in Erdős Problem 249. The crossed arrow is therefore not a missing stylistic qualification: proving it would settle the problem.

One theorem illustrates the second task. Lean has checked a finite certificate at every integer scale $t \le 82$. Here a *certificate* is finite displayed data from which Lean can decisively verify that a particular number is not an integer. The corresponding open requirement asks for certificates beyond every fixed cutoff. Whatever the largest checked scale is, a cutoff lies beyond it, and a finite list says nothing there. The finite theorem therefore does not settle the open problem.

A later README edit can nevertheless overstate the result. By an equivalence proved in the development (Section 3), certificates beyond every fixed cutoff are not merely better evidence. Their existence is equivalent to answering Problem 249 affirmatively: the totient constant is irrational. Three objects recur throughout this paper: the *finite theorem*, certificates at every scale $t \le 82$; the *open requirement*, certificates beyond every cutoff; and the *equivalence* between the open requirement and the irrationality. Lean has checked the first and the third; no Lean theorem carries the first to the second, and proving that implication would settle the problem. An edit that presents the finite cases as completing the open requirement therefore asserts, in English, exactly the bridge the development does not contain; in substance it announces a solution to an open problem. Every Lean proof remains valid under that edit, because the README is not part of any proof. No program in the repository decides whether a line of English has the same meaning as a formal statement. Lean acceptance therefore does not verify the public description.

The design joins five file-and-workflow parts: Lean source, the maintainer-reviewed claim record, authored public documents, generated indexes and summaries, and the release program and continuous-integration workflow. An additional mathematical index helps a reviewer find relevant formal statements, but it has no authority of its own. The repository keeps a *maintainer-reviewed claim record* in `docs/claims.json`. For each selected result the record states the public wording, its status, the named Lean theorems or definitions that support it (Lean calls such named items *declarations*), the bounded domain it covers, and the stronger conclusion the record marks as open. A release checker, `scripts/check_release.py`, then verifies that the recorded relationships still hold across the Lean source, the record itself, the authored public pages, and the generated files. The record covers only *registered claims*, meaning claims entered into it, not every line of prose in the repository; software cannot preserve a relationship it

was never pointed at, and Section 5 shows this limit operating in practice. The shortest accurate summary of the division of labour is:

> Lean checks the formal proofs. A maintainer reviews what those proofs mean and how they may be described. The release machinery checks that the recorded relationships remain intact before a release.

Contribution	**Evidence in this paper**	**Not established**
Preserve a selected, human-reviewed relation between a formal result and public wording.	One implemented claim record, separate proof and release jobs, and one historical escaped edit rejected after registration.	Correct semantic interpretation, complete discovery of claim-bearing prose, a detection rate, independent review, or transfer to another project.

Section 2 separates human review from automatic checking; Section 3 follows one theorem into its public record; and Sections 4–6 state what the checks establish, examine the escaped edit, and delimit reuse. Readers focused on the release design may skip the calculation in Sections 3.1–3.2.

2 The release workflow

Read the upper half of Figure 2 from left to right. A mathematician first reads the Lean source and records the approved public wording and its limit in the claim record. A separate mathematical index helps the reviewer find formal statements, but it is only a generated navigation view and has no authority over the claim. Authored public documents use the reviewed wording; generated views merely reorganise source records. The dashed arrows show the two human judgements. The lower half is automated: each job runs on its own fresh copy. The decision is only whether both jobs pass.

Once a person has compared a formal theorem with its public wording, a program can preserve the resulting decision about names, files, wording, and limits. It cannot decide whether the decision was correct. The workflow does not technically force a second independent mathematician: one maintainer can edit the Lean statement, record, and prose together so that every comparison agrees with the same mistake.

The two automated jobs answer different questions. `lake build` checks the formal statements and proofs; it never reads the README. `python3 scripts/check_release.py` compares the recorded relationships among source, record, public pages, and generated files; it does not run Lean. The workflow in `.github/workflows/lean.yml` runs them as two separate jobs and begins each from a fresh checkout [2]. A checker error stops the program, and the workflow treats that exit as a failing job. A pass proves no mathematics, and publication remains a human decision.

Generated views are rebuilt rather than edited and create no new mathematics. The complete file map, ownership table, and maintenance commands live in `ARCHITECTURE.md`; this paper gives only what this argument needs.

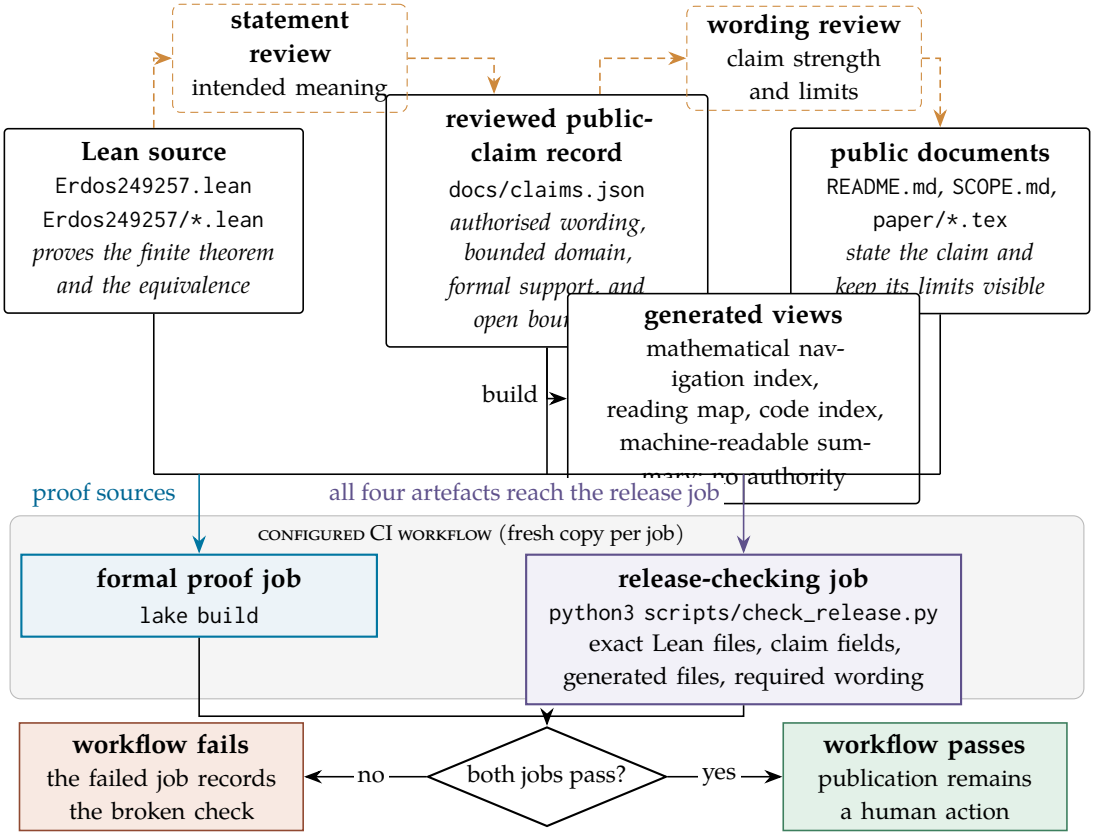

Figure 2: The configured checks for one saved change. Dashed elements are human: one maintainer performed both review steps here. Solid elements are files, programs, and automated checks. Lean checks the formal proofs. Human review compares their intended meaning with authored classifications and the recorded public wording and limits. The release job checks that recorded names, wording, Lean files, and generated views still agree. Continuous integration reruns the Lean and release jobs; neither interprets unrestricted prose. In the worked example of Section 3, the release job protects the recorded distinction between the finite theorem and the open requirement. Blue marks the Lean job, amber review, and violet the release job.

3 One claim from Lean theorem to public page

We now follow one result from Lean source to public page. We begin with its public meaning, not its internal name:

> Lean has checked a finite certificate at every lcm-diagonal scale $t \leq 82$. The registered open requirement asks for certificates beyond every fixed cutoff. This finite interval does not settle it.

3.1 What the certificates are for

Erdős Problem 249 asks whether the totient constant

$$S = \sum_{n \geq 1} \frac{\varphi(n)}{2^n}$$

is *irrational* [3], meaning that it is not a/b for any integers a and b with $b \neq 0$. Here φ is Euler's totient function: $\varphi(n)$ counts the integers from 1 to n sharing no factor greater than 1 with n. The following exact equivalence explains why the certificates matter. Multiplying S by 2^N makes its first N terms integral; write R_N for the remaining rescaled tail. Thus R_N differs from $2^N S$ by an integer. Put $D_{N,h} = R_{N+h} - R_N$. It differs from $2^N(2^h - 1)S$ by an integer. For $h > 0$, the factor $2^N(2^h - 1)$ is a nonzero integer, so an integral $D_{N,h}$ would force S to be rational. Conversely, every rational number has an eventually repeating binary expansion, the base-2 analogue of a repeating decimal. Thus some $h > 0$ makes $D_{N,h}$ integral for every sufficiently large N. Lean checks the exact equivalence:

$$S \text{ is irrational} \quad \Longleftrightarrow \quad D_{N,h} \notin \mathbb{Z} \text{ for every } h > 0 \text{ and every } N.$$

For fixed h and N, Lean proves $D_{N,h} \notin \mathbb{Z}$ exactly when a certificate exists at a finite depth L. Its integer calculation approximates $2^L D_{N,h}$ with error at most $r = N + h + L + 2$. The certificate checks that the residue modulo 2^L lies strictly between r and $2^L - r$, outside both bands compatible with an integral value. The omitted infinite tail is thus accounted for by an explicit bound. The claim record calls this a *certified kill*. We use the shorter word *certificate*: it proves only that this particular tail difference is not an integer.

The scales of the finite theorem compress this two-parameter requirement to one. The diagonal $H_t = \operatorname{lcm}(1, \ldots, t)$ is the least common multiple of $1, \ldots, t$: the smallest positive integer divisible by every integer in that list. It is therefore divisible by every candidate period up to t. If S were rational, its binary digits would have a period h_0 after a position N_0. For $t \geq \max(h_0, N_0)$, one has $h_0 \mid H_t$ and $H_t \geq N_0$, so rationality would make D_{H_t,H_t} integral; a diagonal certificate rules this out. The development proves the reduction exact: certificates along the diagonal beyond every fixed cutoff are equivalent to the full two-parameter requirement, and therefore, through the chain above, to the irrationality itself. Certificates on a finite list, however long, are not.

Writing $\mathrm{Cert}(t)$ for "a certificate exists at scale t", the two statements then read:

$$\begin{aligned} \text{checked:} &\ \mathrm{Cert}(t) \text{ for every integer } t \leq 82; \\ \text{open:} &\ \text{for every cutoff } T,\ \mathrm{Cert}(t) \text{ for some } t > T. \end{aligned}$$

The boundary between them is a matter of logical form, not of scale. The bound 82, 820, or any other fixed bound stands in the same relation to the open requirement: a larger cutoff still exists. No finite interval implies the second statement without a theorem that produces new witnesses. In particular, no certificate at $t = 83$, and no cofinal supply, is claimed. This is the boundary an edit can silently delete.

3.2 The formal evidence

The historical module `Erdos249257/DiagonalPincerCertificatesT64.lean` deposits certificates at 28 prime-power breakpoints through $t = 64$. Because H_t is constant between breakpoints, its combined theorem already covers every $t \leq 66$. The later module `ErdosProblems/Skip/LadderT67.lean` adds the remaining finite arithmetic and proves `exists_diagonalKill_le_82`: for every $t \leq 82$, there is a checking depth and a certificate at period and position H_t. The claim record names six supporting declarations across the two Lean libraries.

Lean's *kernel* is the small trusted program that checks every accepted proof. These finite certificate proofs use `decide`, which evaluates a finite proposition and supplies a proof term for the kernel [4]. Separately, `Erdos249257/LcmConeFlatness.lean` proves the exact tail-difference, pointwise-certificate, and diagonal equivalences, ending at `irrational_totient_series_iff_lcm_diagonal_certificate_supply`. The default `lake build` compiles the libraries rooted at `Erdos249257.lean` and `ErdosProblems.lean`, including the module that proves the $t \leq 82$ theorem.

3.3 What "coverage" means here

The word *coverage* needs an object. This project separately tracks which named Lean items exist, where readers can find them, which statements a person has described, and which descriptions were approved for public use. Those are inventory, navigation, interpretation, and public-claim coverage; none implies the next. Lean calls a named theorem or definition a *declaration*. Indexing every declaration is not understanding every theorem, and understanding every theorem would still not find every sentence that mentions one. Dated inventory counts belong in Appendix A; they are navigation facts, not a result.

3.4 The reviewed record

The quoted record entry contains four names specific to this repository. A *certified kill* is the certificate defined in Section 3.1. The entry abbreviates least common multiple as "lcm", so its *lcm-diagonal scales* are the values H_t. The *small periods* are the periods 1 through 8, each handled by one explicit certificate. A *certificate shard* is the same calculation at a fixed period, position, and depth, isolated in another Lean file and checked by `decide`.

Table 1 shows the substance of the corresponding entry in `docs/claims.json`, ordered as a reader meets it: what is claimed, where it stops, what remains open, then the supporting machinery. The entry also records an exact source line for each declaration; those coordinates are omitted from the table.

Field	Content
Public statement	"Lean checks every lcm-diagonal scale $t \leq 82$ and the listed finite shards. The historical 28 deposits through $t = 64$ already covered $t \leq 66$ because the lcm is constant between prime powers."
Bounded domain	Listed small periods, every $t \leq 82$, and the fixed parameters of each finite shard; no $t = 83$ or cofinal claim.
Remaining open	A link to the registered open requirement *unbounded certificate supply*: produce a certified witness beyond every fixed cutoff.
Status	*verified finite instance*: Lean checked the stated finite inputs.
Supporting declarations	Six named Lean theorems with recorded modules and source lines, ending in `exists_diagonalKill_le_82`.
Claim identifier	`certified_kill_instances`

Table 1: The reviewed claim entry for the worked example, exact source coordinates omitted. Together, the positive statement, bounded domain, and remaining-open link distinguish the finite result from the open problem. In the failure of Section 5, overstatement came from deleting this boundary, not from inventing a theorem.

The table records rather than explains the claim. Its public statement, bounded domain, remaining-open link, and status record a human judgement about meaning. Under the proved equivalence, the remaining-open field is essential: without it, the entry no longer separates the finite theorem from an answer to Problem 249. The identifier and source lines instead help software find the entry and may be refreshed when code moves.

No field establishes that the English is faithful to Lean. The maintainer read the final theorem, judged that the wording claims the listed scales and no more, and confirmed that the unbounded requirement remains open. The record stores that judgement, not its justification. Review asks whether wording is faithful; authorisation decides what the project publishes. One maintainer performed both here. A later change to either theorem or wording reopens the review; automatic checks can only preserve the decision that was recorded.

3.5 What the checker verifies here

For this claim, `scripts/check_release.py` verifies a finite list of relationships. Declaration names must remain near their recorded source locations, and all checked Lean source must byte-match the saved revision. The limitation clause must remain on each registered public page, and generated views must equal a fresh rebuild. These tests establish presence, identity, and proximity, not implication. Lean checks the theorem; the release program checks the recorded description around it; a person remains responsible for the claim that the two say the same thing.

Layer	What it can establish	What it cannot establish
Lean build and kernel	The written formal statements are proved from their imported definitions and assumptions, using the recorded Lean version and dependencies.	That a formal statement is the one the author intended, or that any English description of it is faithful.
Maintainer review	A recorded judgement that a named formal statement has the intended meaning and selected public wording describes it within stated limits.	That Lean accepts the proofs; that every important claim was registered; or that the review was independent or correct.
Release checker	That recorded names, source lines, fields, links, required wording, Lean files, and generated files satisfy the declared rules; that prohibited proof shortcuts are absent; and that deliberately wrong examples still fail.	What unregistered prose means; whether the record is complete; or whether the human judgements it preserves are correct.
Continuous integration	That each job ran on its own fresh copy of the uploaded revision and exited successfully.	Independent mathematical approval; anything beyond what the two jobs themselves establish.

Table 2: What each layer establishes and leaves open.

4 What the checks establish

The word "verification" is easy to overread here. Four separate questions are involved. Did Lean accept the formal statements and proofs? Did a maintainer record a judgement that selected public wording describes them within stated limits? Do the configured structural comparisons pass at this revision? Did continuous integration run the configured jobs on that revision and report success? Each question needs different evidence. A yes to one does not settle the others.

When both jobs pass, Lean has accepted the proofs and each configured structural comparison has passed. This does not repeat the maintainer's review, establish that the review was correct, or show that every consequential claim was registered. Table 2 states what each layer can and cannot establish. Keeping the layers separate prevents the single word "verified" from making their combined guarantee sound stronger than it is.

Instantiated on the worked example, the source proves the finite theorem and equivalence, not the open requirement. Maintainer-reviewed wording confines the theorem to the listed scales. The release checker preserves declaration coordinates, revision agreement, the limitation clause, and rebuilt views; continuous integration reruns the jobs on fresh copies.

The limits follow the same lines. Lean cannot notice English that quietly implies the open requirement met. The recorded review does not show that every page repeating

the claim was found. The checker cannot reject a paraphrase it was never given. Passing both jobs cannot make a mistaken reading of the equivalence correct.

The release checker rejects proof placeholders, project-defined axioms, and native evaluation; ordinary `decide` produces a kernel-checked proof term [4, 5]. It compares reviewed sources, rebuilds views, verifies paper hashes, and reruns selected deliberately false examples (called *negative fixtures* in the repository). These checks preserve recorded relationships, not mathematical meaning or unseen faults.

For a mathematical change, run `lake build`; review statements, assumptions, and meaning; regenerate views; then run `python3 scripts/check_release.py`. Continuous integration reruns the two separate jobs. Prose-only edits still need human and release review; stronger wording reopens the judgement. The repository guide lists the full commands.

5 A boundary the checklist missed

The evidence comes from three different times. The historical exercise, the present post-repair test, and the later executable reconstruction answer different questions and must not be merged. Appendix A identifies their files and commands.

Historical exercise. The structured report in `docs/publication_evidence.json` identifies a saved Git revision. It says that ten deliberate false edits were applied to a separate copy one at a time and ran the release checker, but not Lean. They covered seven configured relationship kinds, including status, source coordinates, boundary wording, generated-file freshness, and size budgets. The original run logs were not retained; the file is a report, not raw output.

According to that record, nine of the ten edits were rejected. One escaped: the README clause saying that the finite cases *do not supply* the open requirement was changed to say that they *complete* it, asserting exactly the missing bridge to Problem 249. Lean was untouched, and the checker passed because the clause was absent from its checklist. The escape shows that a passing checker does not certify all public prose; it gives no detection rate.

Repair and present test. The repair required the clause and added a deliberately false example, called a *boundary witness* in the repository: it crosses a consequential public boundary while leaving every Lean proof intact. The post-repair witness accepts the current README and rejects a test copy containing the false clause. The other nine edits were not rerun against the extended checklist, so there is no post-repair aggregate result.

Later reconstruction. The executable reconstruction preserves three original targets and uses seven documented replacements. It is a new experiment, not the missing runs.

The evidence marks a coverage boundary, not a reliability score. The edits were authored by the checker's author; all seven relationship kinds were represented, but five of them by one edit each; and no controlled comparison with disciplined manual review was run. Reader error, ordinary use, and transfer were not measured.

The retained evidence supports one design lesson:

> The program can preserve a relationship only after a person has identified and recorded that relationship.

Requiring a clause to be present does not detect a contradictory stronger claim elsewhere on the page. Coverage grows only when a person notices and records a relationship; selected high-risk commitments also have a negative fixture, that is, a deliberately false version which the check must reject. Coverage can also shrink: deleting a registered relationship leaves no rule requiring it, so every remaining check passes. Retiring a commitment therefore deserves the same review as creating one.

The shape of this limit is familiar from Section 3.1. A configured checklist, like a finite certificate interval, does not exhaust an open-ended domain. The comparison is only structural: prose has no equivalence theorem and no diagonal compresses it.

Three boundaries the companion papers exposed. Three later corrections fell outside the checklist for different reasons. A kernel-checked headline in the Problem 249 note had no claim-record entry, so it carried no reviewed public status. A Problem 257 support described as open had been settled in the literature in 2019, outside the repository. A valid parity theorem was presented as a frontier until adjoining one support element was seen to remove the obstruction: the theorem was correct but its advertised significance was not. These are respectively a registration gap, stale external status, and representation-dependent significance. They are incidents, not a rate, and none is detectable by comparing recorded artefacts with one another.

6 Scope, reuse, and limits

The failures the design addresses. The checks address accidental disagreement after a sound review. A declaration may move while its recorded line number stays fixed, and a generated view may be stale or hand-edited. A rewrite may remove a required limitation, a status may be upgraded, or the shipped Lean files may cease to be the reviewed ones. A check may also decay until it can no longer fail. In each case, one recorded item disagrees with the others, which a mechanical comparison can detect.

The failures it does not address. Three failure modes remain outside the design. *Unregistered wording:* a paraphrase can strengthen a claim without touching a registered anchor, and a contradiction, misleading emphasis, or new public document can escape for the same reason. Requiring one clause to be present does not require the page to agree with it. *Coordinated but wrong change:* a maintainer can alter source, record, and prose together, so every comparison agrees. If the certificate definition were weakened while the record and prose were updated to match, all checks could pass although the public reading of the equivalence was wrong. *Mistaken review:* if the original judgement was wrong, the machinery preserves the mistake. It checks persistence, not the quality of the judgement.

The checked boundary. Only listed relationships are checked. The repository calls their set its *registered checking boundary* (historically, its "assurance perimeter"); in ordinary terms, this is simply the boundary of the checklist. A recorded commitment inside the boundary triggers an automatic check. Outside it, the checker is silent. The name must not suggest more: choosing what belongs on the checklist remains a human judgement, and a passing check does not make the contents true. This repository gives priority to headline results and to wording whose accidental strengthening would change the project's public status, as the deleted boundary clause did under the equivalence. If the same claim appears on several pages, the checker sees only the appearances named in the record.

Several narrower limits belong to this implementation rather than to the general pattern. One maintainer performs both review and authorisation; the checker tolerates a theorem name within three lines of its recorded location; some prose checks require exact wording; the exercise used one edit per relationship type; and the original logs were not retained.

What another project could reuse. The file formats are incidental. The pattern applies whenever a checked formal result sits beside a tempting stronger public statement: finitely many cases beside a statement about all cases; a conditional theorem beside its unproved hypothesis; one implication beside an alleged equivalence; or a theorem under assumptions beside an unconditional headline. What transfers is the reviewed boundary between the exact formal result and its nearest unproved strengthening.

The minimum obligations are concrete. Each selected public claim needs its own wording, named formal evidence, exact source version, scope, assumptions, and explicit stronger non-conclusions. Human mathematical review must be distinguished from automatic consistency checking. Every generated public view needs a named builder. Proof checking and publication checking must be separate and able to fail independently. Every recorded relationship needs a specified mechanical check; selected high-risk boundaries should also have a deliberately false example which that check rejects. Finally, the project must describe its selection as selective rather than complete.

What is not established. The architecture does not establish a solution to either Erdős problem; that a formal statement is the statement the author intended; that software understood any unregistered prose; that the record is complete; that one maintainer's review is independent or adequate; or that the design transfers to other projects with its behaviour intact. A large number of passing comparisons measure none of those things. What a passing workflow establishes is narrower: the configured jobs ran on the named revision, Lean accepted the formal proofs, and every configured structural comparison passed. This paper is an architecture note with one bounded case study and three naturalistic incidents, not an empirical evaluation of the design. A credible evaluation would need review records naming reviewer and revision, wrong edits authored by someone other than the checker's author, and only then a comparison with ordinary review on the same changes. None of those stages exists here, and this paper claims none of them.

7 Related systems

The nearest systems divide into four groups, each solving a different part of the problem. Proof blueprints connect an informal proof plan to Lean declarations. Semantic-review tools help people compare formal statements with intended mathematics. Checked-document and traceability systems record relationships among heterogeneous artefacts. Software-testing methods probe whether selected checks fail when they should. This repository borrows from all four but replaces none of them: its narrower purpose is to preserve, after review, the boundary between an accepted formal result and a recurring public claim.

Proof blueprints. A *proof blueprint* pairs an informal outline with Lean declarations and uses author-supplied links to record which results depend on which earlier results. `leanblueprint` stores that outline in TEX and checks that every named declaration exists [6]. LeanArchitect attaches blueprint metadata to Lean source, infers dependencies and unfinished-proof status, and exports synchronised TEX [7]. Text and unformalised nodes remain human responsibilities. Both systems primarily support a formalisation in progress. The claim record here instead starts after a selected proof has been accepted and asks which public wording was reviewed and which stronger conclusion remains open.

Reviewing intended mathematical meaning. Lean Atlas and EconCSLib ask whether a formalisation expresses its intended source mathematics. Lean Atlas leaves semantic verification to people. Given chosen theorem statements, its Lean Compass selects the project declarations whose meaning can affect them, narrowing what a person must inspect; it assumes Lean's standard library and the mathematical library Mathlib are semantically correct rather than inspecting them [8, Algorithm 1 and Proposition 4, p. 6]. The authors' soundness claim is conditional on the semantic correctness of every returned declaration and the trusted base; project-level coverage additionally requires the chosen theorem set to exhaust the intended claims. It does not find every claim occurrence in later public prose. EconCSLib instead has a model write Lean. At statement level, one context-blind model pass translates Lean back to LATEX and a second compares that translation with the source; a separate holistic audit checks cross-statement drift, and a dashboard records human judgements. At present all model validations run through one Codex agent stack, so errors may correlate; saved human review remains sparse and incomplete [9, §3.2.1, pp. 4–7]. This paper addresses a later boundary: it starts from accepted Lean proofs and asks how subsequent public wording can remain within a reviewed interpretation of them.

An audit of formal-theorem benchmarks likewise finds that kernel acceptance does not establish fidelity to the intended natural-language problem. Across five benchmarks and 13 released variants, its static checkers produced 4,833 signals, 398 with machine-checkable certificates; a separate semantic audit evaluated 92 labelled natural-language/Lean pairs across six error categories and achieved high recall but low precision, so human adjudication remained necessary [10, abstract; §§4.1–4.3, pp. 6–7; Tables 4–7]. That work concerns incoming benchmark statements; this repository concerns

outgoing claims made after a proof. Its program is narrower than semantic verification and still cannot prove that every claim-bearing passage was selected.

Checked documents and traceability. Isabelle/DOF, a document system built on the Isabelle proof assistant, places formal and informal material in one checked document. Authors define an *ontology*: document classes with typed fields and rules. They label passages with those classes, and Isabelle's editor reports rule violations as they edit [11]. The bounded domain and open conclusion in Table 1 could be typed fields in such an ontology. This repository keeps prose unrestricted and checks a separate record; its checker cannot inspect prose that the record does not name.

Testing the boundary. CASCADE derives tests and an alternative implementation from the same documentation using language models. It reports a likely inconsistency only when at least one generated test fails on the original implementation and passes on the generated one, and no generated test changes in the opposite direction. A person must still confirm the report [12, Algorithm 1 and §3, article pp. FSE168:6–8]. Unlike the present checker, it can inspect documentation that was not registered in advance.

Mutation testing deliberately seeds faults and asks whether tests distinguish the altered program from the original [13, 14]. The ten false edits in Section 5 serve that purpose only. They were selected by hand, and nine were not rerun after repair, so they yield neither a post-repair detection rate nor a mutation-adequacy score.

Requirements traceability follows a requirement through development and revision. Gotel and Finkelstein distinguish its production before a requirements specification from its deployment afterwards [15, §§5.1–5.4]. The claim record is closer to bounded post- specification traceability: it does not reconstruct exploratory requirement production, changing responsibility, or contributor access. An assurance case is a reasoned argument supported by evidence; Goal Structuring Notation (GSN) is one graphical notation for documenting its claims, evidential references, context, and asserted support relationships [16, §§0:2.2–0:4.2, pp. 10–11]. The standard is explicit about the boundary: the notation documents an asserted argument but establishes neither its truth nor that it sufficiently supports the top claim [16, §0:3.2, p. 11; §§0:4.11–0:4.13, p. 15]. The claim record resembles those elements but is not an assurance argument: it contains no chain of reasoning asserting that the Lean declarations justify the public wording. It records the approved wording, source, scope, and limits; the justification remains a human judgement.

8 Conclusion

Lean has checked certificates at every lcm-diagonal scale $t \leq 82$. It has not checked a certificate at $t = 83$ or an unbounded supply. The development proves that an unbounded supply would settle Problem 249, so the difference cannot be dismissed as cautious wording around essentially the same result. Problems 249 and 257 remain open.

The systems result is correspondingly modest and useful. A mathematician can record the exact public wording authorised by a formal result, the result's range, and its nearest open strengthening. A release program can then reject later edits which violate

those recorded relationships. In this repository a deliberately false test case—called a negative fixture in the repository—now catches the strengthening that historically escaped.

The failure supplies the governing limit. The registered checking boundary is only as wide as the relationships someone chose to record; in ordinary language, the checker cannot enforce an item absent from its checklist. Retiring an item is therefore as consequential as adding one and deserves the same review. The GSN development guidance makes the corresponding repair concrete: if evidence does not cover the lowest claim, state the claim that it actually supports, weaken or bound that claim, and revisit the claims above it [16, §2:3.8.1, p. 70]. The method preserves selected mathematical judgements after they have been made. It does not make those judgements correct, discover every public claim, provide a detection rate, or establish transfer to another repository. A passing workflow says only that Lean accepted the formal proofs and the configured structural comparisons passed on the named revision.

A Reproducibility

Dated navigation counts. At the reviewed snapshot, all 151,755 live declarations were inventoried and routed, and all 141,820 author-written theorem-like declarations had an exact node link. Of those, 139,370 (98.3%) participated in authored mathematical interpretations: 2,895 as exact proposition evidence and 136,475 as bounded certificate- or module-family context. The remaining 2,450 were linked only through exact source-module and normalised-signature families, not authored mathematical paraphrases. Every declaration selected for a public claim had an authored route. The command `python3scripts/query_semantic.pycoverage` derives these volatile navigation counts and checks their references. They do not measure semantic review quality or public-claim completeness.

The reviewed claim record, `docs/claims.json`, names the saved Git revision of the Lean source, which the release checker requires to match exactly. This paper omits the changing commit identifier so that the claim record is the only place that states it.

Declaration of generative AI use. Large-language-model agents were used throughout development to draft and revise prose, formal proofs, and software. The author set the objectives and acceptance criteria, selected and reviewed the claims, and approved the published version. The author assumes responsibility for the accuracy, interpretation, and presentation of the work. Generative systems are production tools; they are not authors and supply no independent authority. That boundary is the subject of this paper as well as a condition of it: the checker described in Section 4 tests recorded relationships a person selected, so passing it does not make generated wording faithful. The author selected, reviewed, and authorised the public claim-to-declaration mappings, and remains responsible for connecting the proofs to public wording.

The paper inventory, `docs/publication_contract.json`, records source and PDF cryptographic hashes and validation commands. The evidence file for Section 5, `docs/publication_evidence.json`, records the protocol, outcomes, and limitations. The reconstruction file, `experiments/publication_mutations.json`, specifies the ten edits.

The script `scripts/run_publication_mutations.py` checks that each edit applies once (`--verify-operators`) and, by default, runs all ten in a copy of the saved evaluation version (`--all`). For an edit whose exact original target was not preserved, the file names a fixed replacement. The original raw outputs were not retained; the reconstruction does not claim to be them.

The papers build with Tectonic or standard LaTeX via `make -C paper`. The tracked root PDF is the shipped copy, and the inventory checks its file hash. These identities name artefacts; they do not interpret them.

References

[1] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, in *Automated Deduction—CADE 28*, Lecture Notes in Computer Science 12699, 2021, pp. 625–635, DOI.

[2] GitHub, *GitHub-hosted runners*, documentation, accessed 18 July 2026.

[3] P. Erdős and R. L. Graham, *Old and New Problems and Results in Combinatorial Number Theory*, Monographies de L'Enseignement Mathématique 28, L'Enseignement Mathématique, Université de Genève, 1980, p. 61, scan.

[4] Lean project, *The Lean Language Reference: Validating a Lean Proof*, documentation, accessed 18 July 2026.

[5] Lean project, *The Lean Language Reference: Axioms*, documentation, accessed 18 July 2026.

[6] P. Massot, *leanblueprint*, plasTeX plugin for Lean formalisation blueprints, 2020, software repository, accessed 18 July 2026.

[7] T. Zhu, P. Monticone, S. Welleck, and J. Avigad, *LeanArchitect: Automating Blueprint Generation for Humans and AI*, in *17th International Conference on Interactive Theorem Proving*, LIPIcs 382, 2026, pp. 25:1–25:16, DOI.

[8] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, arXiv:2604.16347.

[9] N. Garg, *EconCSLib: AI-Assisted Lean Formalization for Economics & Computation Research*, 2026, arXiv:2606.13306.

[10] P. S. Ammanamanchi, S. Bhat, and S. Biderman, *Faults in Our Formal Benchmarking: Dataset Defects and Evaluation Failures in Lean Theorem Proving*, in *Proceedings of the 43rd International Conference on Machine Learning*, PMLR 306, 2026, arXiv:2606.29493.

[11] A. D. Brucker and B. Wolff, *Isabelle/DOF: Design and Implementation*, in *Software Engineering and Formal Methods*, Lecture Notes in Computer Science 11724, 2019, pp. 275–293, DOI.

[12] T. Kiecker, J. A. Sparka, M. Reuter, A. Ziegler, and L. Grunske, *CASCADE: Detecting Inconsistencies between Code and Documentation with Automatic Test Generation*, *Proceedings of the ACM on Software Engineering* 3 (FSE), Article FSE168, July 2026, 23 pages, DOI.

[13] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, *Hints on test data selection: Help for the practicing programmer*, IEEE Computer 11(4), 1978, pp. 34–41, DOI.

[14] Y. Jia and M. Harman, *An analysis and survey of the development of mutation testing*, IEEE Transactions on Software Engineering 37(5), 2011, pp. 649–678, DOI.

[15] O. C. Z. Gotel and A. C. W. Finkelstein, *An analysis of the requirements traceability problem*, in *Proc. First IEEE International Conference on Requirements Engineering*, 1994, pp. 94–101, DOI.

[16] SCSC Assurance Case Working Group (ACWG), *Goal Structuring Notation Community Standard, Version 3*, SCSC-141C, May 2021, standard.